Guía de Actividades **Práctico-Experimentales** Nro. 003

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Integrantes	Matias Encarnación Matias Lavanda Julián Merino Sherman Abarca Jefferson Sarango
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	03
Título de la Práctica	Mapeo de referencias en memoria y la auditoría de fugas de memoria.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 15 y viernes 16 de octubre 2026
Horario	Jueves 15 de 08:30 – 10:30 Viernes 16 de 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Silabo	3 horas

2. Objetivo(s) de la Práctica:

- Modelar el ciclo de vida de los objetos en memoria, diferenciando físicamente la asignación estática (Stack) de la asignación dinámica (Heap) dentro del motor V8 de JavaScript.

- Auditar y cuantificar empíricamente el desperdicio de memoria (fugas o Memory Leaks) originado por referencias colgantes y cierres léxicos mal gestionados.

- Aplicar principios de Green Computing induciendo manualmente el Recolector de Basura (Garbage Collector) para medir el ahorro de memoria RAM y el decrecimiento de la complejidad espacial $S(n)$ a lo largo del tiempo.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para investigación en documentación oficial de V8.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El servidor satelital desarrollado en la Práctica 001 lleva funcionando 48 horas continuas procesando coordenadas, pero repentinamente colapsa mostrando el error FATAL ERROR: Ineffective mark-compacts near heap limit Allocation failed - JavaScript heap out of memory. El equipo debe auditar el código, encontrar por qué la memoria crece a un ritmo de $O(n)$ sin límite, y aplicar una solución arquitectónica para estabilizar el consumo.

- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo opera el algoritmo *Mark-and-Sweep* (Marcado y Barrido) del *Garbage Collector* (GC) y por qué las referencias globales impiden su ejecución.
- **Modalidad:** Individual.

Tarea 1: Simulación de una Fuga de Memoria (Memory Leak)

Para poder auditar un problema de memoria, primero debemos crearlo modelando un historial de ubicaciones satelitales que jamás se descarta.

1. En el directorio de su proyecto, cree un archivo llamado lab03_memory_leak.js.
2. Implemente el siguiente código que simula la llegada asíncrona de datos que se acumulan en el entorno global (Heap).

```
const { performance } = require('perf_hooks');

// Referencia global: Actúa como raíz (root), el GC no puede eliminar lo que esté aquí.
const historialSatelitalGlobal = [];

console.log("[Auditoría] Iniciando simulación de Memory Leak...");

function procesarCoordenada() {
    // Simulamos un paquete de datos entrante (aprox. varios kilobytes)
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        // Carga útil pesada para acelerar el llenado de la memoria
        payload: new Array(100000).fill('datos_topograficos_basura')
    };
   // El error lógico: Almacenamos todo indefinidamente causando S(n) = O(n)
    historialSatelitalGlobal.push(paquete);
}

// Monitoreo de la memoria Heap cada 100 iteraciones
for (let i = 1; i <= 500; i++) {
    procesarCoordenada();
   if (i % 100 === 0) {
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 /
1024);
        console.log(`Iteración ${i} | RAM consumida: ${memoryUsed} MB`);
    }
}
```

```
console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} registros en el Heap.`);
```

3. Ejecute el script con node lab03_memory_leak.js y observe cómo la RAM crece linealmente.

Tarea 2: Forzado del Recolector de Basura (Garbage Collector)

Por defecto, el GC de Node.js es no determinista. Para medir si estamos solucionando la fuga, forzaremos su ejecución.

1. Duplique el archivo anterior y nómbrelo lab03_gc_force.js.
2. Modifique el ciclo for para invocar al Recolector de Basura. Node.js requiere que esto se haga mediante una bandera especial de ejecución.

```
// Añada esto dentro de su ciclo iterativo, después del push al historial
if (i % 100 === 0) {
    // Si global.gc está expuesto, lo forzamos a limpiar
    if (global.gc) {
        global.gc(); // Ejecutamos el barrido manual
    }
    const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
    console.log(`Iteración ${i} | RAM post-GC: ${memoryUsed} MB`);
}
```

3. **Ejecución estricta:** Para exponer la función gc(), ejecute en terminal: node --expose-gc lab03_gc_force.js.
4. **Análisis:** Notará que la RAM sigue creciendo. El *Garbage Collector* no eliminó la basura porque el arreglo global historialSatelitalGlobal mantiene una **referencia activa** hacia los objetos. El algoritmo *Mark-and-Sweep* los marca como "en uso".

Tarea 3: Resolución y Mantenimiento Ecológico (O(1) Espacial)

1. Cree el archivo lab03_solucion.js.
2. Refactorice la lógica eliminando referencias obsoletas, permitiendo que el GC las reclame, manteniendo un historial cíclico de máximo 50 registros espaciales.

```
const historialSatelital = [];
const LIMITE_HISTORIAL = 50; // Constante C, por lo que S(n) = O(1)

function procesarCoordenadaOptimizada() {
    const paquete = {
```

```
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        payload: new Array(100000).fill('datos_topograficos_optimizados')
    };

    historialSatelital.push(paquete);

    // Desvinculación léxica: Eliminamos la referencia al objeto más antiguo
    if (historialSatelital.length > LIMITE_HISTORIAL) {
        historialSatelital.shift(); // Quitamos el primer elemento (FIFO)
    }
}

for (let i = 1; i <= 1000; i++) {
    procesarCoordenadaOptimizada();

    if (i % 200 === 0) {
        if (global.gc) global.gc(); // Forzamos el GC
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 /
1024);
        console.log(`Iteración ${i} | RAM Controlada: ${memoryUsed} MB`);
    }
}
```

3. Ejecute: node --expose-gc lab03_solucion.js. Compruebe cómo el consumo de memoria alcanza un techo y se mantiene estable, resolviendo el colapso del servidor.

6. Resultados esperados:

- Los estudiantes lograrán observar de manera tangible cómo una referencia activa en una estructura de datos (Arreglo) impide la liberación de memoria dinámica (*Heap*). Aplicando un límite lógico, lograrán transformar un consumo de complejidad $S(n) = O(n)$ (infinito en el tiempo) a uno constante $S(n) = O(1)$, aplicando principios de diseño energéticamente eficientes.

  Código Tarea 1:

```js
const { performance } = require('perf_hooks');

// Referencia global: Actúa como raíz (root), el GC no puede eliminar lo que esté aquí.
const historialSatelitalGlobal = [];

console.log("[Auditoría] Iniciando simulación de Memory Leak...");

function procesarCoordenada() {
    // Simulamos un paquete de datos entrante (aprox. varios kilobytes)
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        // Carga útil pesada para acelerar el llenado de la memoria
        payload: new Array(100000).fill('datos_topograficos_basura')
    };
   // El error lógico: Almacenamos todo indefinidamente causando S(n) = O(n)
    historialSatelitalGlobal.push(paquete);
}
```

```js
// Monitoreo de la memoria Heap cada 100 iteraciones
for (let i = 1; i <= 500; i++) {
    procesarCoordenada();
    if (i % 100 === 0) {
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM consumida: ${memoryUsed} MB`);
    }
}
console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} re
```

Resultados Tarea 1:

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)> node "c:\Users\nicof\OneDri
ve\Desktop\Archivos\Estructura de Datos (Grupal)\APE(Semana3)\lab03_memory_leak.js"
[Auditoría] Iniciando simulación de Memory Leak...
Iteración 100 | RAM consumida: 80 MB
Iteración 200 | RAM consumida: 157 MB
Iteración 300 | RAM consumida: 233 MB
Iteración 400 | RAM consumida: 309 MB
Iteración 500 | RAM consumida: 385 MB
[Auditoría] Finalizado. El arreglo retiene 500 registros en el Heap.
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)>
```

Código Tarea 2:

```
const { performance } = require('perf_hooks');

// Referencia global que utilizaremos para almacenar y limpiar datos
let historialSatelitalGlobal = [];

console.log("[Auditoría] Iniciando simulación con gestión de memoria...");

function procesarCoordenada() {
    // Simulamos un paquete de datos pesado
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        // Carga útil pesada para acelerar el llenado de la memoria
        payload: new Array(100000).fill('datos_topograficos_basura')
    };
    // El error lógico: Almacenamos todo indefinidamente causando S(n) = O(n)
    historialSatelitalGlobal.push(paquete);
}

// Monitoreo y limpieza de la memoria Heap cada 100 iteraciones
for (let i = 1; i <= 500; i++) {
    procesarCoordenada();
    if (i % 100 === 0) {
        const memoryBefore = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM antes de limpiar: ${memoryBefore} MB`);
        //Vaciamos el arreglo para que los objetos
        //dejen de ser alcanzables desde la raíz global.
        historialSatelitalGlobal = [];
        // FORZAR EL RECOLECTOR DE BASURA
        if (global.gc) {
            global.gc(); // Barrido manual
        }

        const memoryAfter = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM post-GC (liberada): ${memoryAfter} MB`);
    }
}

console.log(`[Auditoría] Finalizado con éxito.`);
```

Resultados Tarea 2:

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)> node --expose-gc "c:\Users\
nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)\APE(Semana3)\lab03_gc_force.js"
[Auditoría] Iniciando simulación con gestión de memoria...
Iteración 100 | RAM antes de limpiar: 80 MB
Iteración 100 | RAM post-GC (liberada): 4 MB
Iteración 200 | RAM antes de limpiar: 80 MB
Iteración 200 | RAM post-GC (liberada): 4 MB
Iteración 300 | RAM antes de limpiar: 80 MB
Iteración 300 | RAM post-GC (liberada): 4 MB
Iteración 400 | RAM antes de limpiar: 80 MB
Iteración 400 | RAM post-GC (liberada): 4 MB
Iteración 500 | RAM antes de limpiar: 80 MB
Iteración 500 | RAM post-GC (liberada): 4 MB
[Auditoría] Finalizado con éxito.
```

Código Tarea 3:

```js
const historialSatelital = [];
const LIMITE_HISTORIAL = 50; // Constante C, por lo que S(n) = O(1)

function procesarCoordenadaOptimizada() {
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        payload: new Array(100000).fill('datos_topograficos_optimizados')
    };

    historialSatelital.push(paquete);

    // Desvinculación léxica: Eliminamos la referencia al objeto más antiguo
    if (historialSatelital.length > LIMITE_HISTORIAL) {
        historialSatelital.shift(); // Quitamos el primer elemento (FIFO)
    }
}

```

```js
for (let i = 1; i <= 1000; i++) {
    procesarCoordenadaOptimizada();

    if (i % 200 === 0) {
        if (global.gc) global.gc(); // Forzamos el GC
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM Controlada: ${memoryUsed} MB`);
    }
}

```

Resultados Tarea 3:

```
[Auditoría] Finalizado con éxito.                                    node --expose-gc "c:\Users\
nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)\APE(Semana3)\lab03_solucion.js"
Iteración 200 | RAM Controlada: 42 MB
Iteración 400 | RAM Controlada: 42 MB
Iteración 600 | RAM Controlada: 42 MB
Iteración 800 | RAM Controlada: 42 MB
Iteración 1000 | RAM Controlada: 42 MB
```

7. Preguntas de Control:

Desde una perspectiva filosófica profunda y analítica, responda en su informe:

- **Causa Material: Al declarar un objeto complejo dentro de una función, ¿en qué "lugar material" (Stack o Heap) se asienta su estructura principal, y en qué lugar se asienta el puntero que lo referencia?**

  Cuando se declara un objeto complejo dentro de una función, su estructura principal y voluminosa se asienta físicamente en el Heap debido a su naturaleza dinámica e impredecible en tamaño, mientras que el puntero o referencia que lo apunta se aloja en el Stack dentro del marco de pila de la función en ejecución, cumpliendo el rol de una coordenada efímera que perece al finalizar el ámbito y deja al objeto huérfano si no existen otras referencias activas que lo anclen.

- **Causa Formal: El contexto global de JavaScript tiene una "forma" que abarca todo el ciclo de vida del programa. ¿De qué manera la estructura formal del alcance (*scope*) global actúa como una jaula que le prohíbe actuar al Recolector de Basura?**

  El alcance (scope) global actúa como una jaula infranqueable para el Recolector de Basura porque su estructura formal está diseñada para perdurar durante todo el ciclo de vida del programa, convirtiéndose perpetuamente en la raíz primaria (root) del grafo de referencias del motor V8; de este modo, cualquier objeto o estructura de datos que mantenga un enlace directo o indirecto con este entorno global es clasificado de manera inalterable por el algoritmo Mark-and-Sweep como activo e indispensable, prohibiendo su eliminación sin importar el tiempo transcurrido o su obsolescencia lógica.

- **Causa Eficiente: El algoritmo de *Mark-and-Sweep* actúa como un destructor activo. Sin embargo, en un *Memory Leak*, ¿quién o qué es el agente responsable (causa eficiente) que detiene la hoja de la guadaña del Recolector de Basura?**

  El agente responsable que detiene la acción del Recolector de Basura durante una fuga de memoria no es una falla del motor de JavaScript ni del algoritmo Mark-and-Sweep, sino la propia lógica de diseño del programador al entablar y sostener referencias activas e ininterrumpidas como un arreglo global que anclan los objetos a la raíz del sistema, obligando al algoritmo a identificarlos falsamente como "en uso" y neutralizando así su capacidad de destrucción.

- **Causa Final: Si el hardware permite almacenar Gigabytes de información temporal, ¿cuál es el propósito final (telos) de programar una destrucción intencional de los datos que nosotros mismos hemos creado con tanto esfuerzo computacional?**

  El propósito final (telos) de programar una destrucción intencional de los datos radica en trascender la ilusión de una capacidad de hardware infinita para garantizar la sostenibilidad operativa, la eficiencia energética bajo principios de Green Computing y la estabilidad a largo plazo del sistema, evitando que la acumulación desmedida de información obsoleta sature el espacio dinámico y provoque el colapso inevitable del software.

- **Esencia y Existencia: Si un paquete de coordenadas queda "olvidado" en el último índice de un arreglo sin que ninguna función lo vuelva a consultar jamás, físicamente ocupa electrones en la RAM, pero lógicamente es**

inaccesible. ¿Existe este dato en la realidad del software o ha perdido su esencia al perder su utilidad? Argumente.

El dato ha perdido su esencia al carecer de utilidad funcional y teleológica dentro del programa, convirtiéndose en basura informática; aunque mantenga una existencia física residual y transitoria debido a los electrones atrapados en la RAM y a la persistencia de una referencia colgante que impide su purga inmediata, ha dejado de ser un elemento real del software para transformarse en un mero residuo inútil que degrada la estabilidad del sistema hasta que el Recolector de Basura rectifique su estado.

8. Declaración de IA

Dentro de la realización del trabajo se hizo uso de Gemini 3.8 Flash para complementar el análisis de los ejercicios propuestos, recibir retroalimentación del funcionamiento de cada método usado para trabajar con los datos y su ciclo de vida con el recolector de basura.

9. Link repositorio de GitHub:

https://github.com/Jullxn28/Ciclo3_EstructuraDeDatos_Practica

10. Bibliografía

[1] P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023..Available: https://open.palni.org/

[2] Mozilla Developer Network, "Memory management and Garbage Collection in JavaScript", *MDN Web Docs*, 2023. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management

[3] Node.js Foundation, "V8 JavaScript engine and CLI options", *Node.js API*, 2023. Available: https://nodejs.org/docs/latest-v20.x/api/cli.html